

قاصدک در برابر جاسوس

شبیه‌سازی پویا و چالش حریم خصوصی در شبکه — پروژه پایانی درس شبکه‌های کامپیوتری (۴۰۴۴۳)

استاد درس: دکتر سید امیرمهدی صادقرزاده — نیمسال دوم ۱۴۰۴-۱۴۰۵

خلاصه نتایج. یک شبکه فضایی با 22 گره (هر گره یک پردازنده مستقل روی یک پورت UDP جداگانه) و 4 کلاستر ساخته شد. در پخش عمومی ساده، حمله پیشنهادی با تنها 2 گره جاسوس (9.1% شبکه) مبدأ 61.4% بسته‌ها را درست تشخیص می‌دهد، در حالی که خط پایه «اولین جاسوس» تنها 39.6% دقت دارد؛ با 6 جاسوس (سقف مجاز ۳۰٪) دقت به 96.8% می‌رسد. پروتکل Dandelion این دقت را به شدت کاهش می‌دهد: با همان 6 جاسوس و $p = 0.9$ ، دقت حمله آگاه از Dandelion به 43.6% می‌رسد — به قیمت افزایش زمان پوشش از $s \ 0.835$ به $s \ 1.545$.

۱. معماری سیستم و قواعد شبکه

۱.۱ معماری داخلی گره

هر گره یک پردازنده مستقل سیستم عامل است (`python -m src.node <config>`) که کنترل‌کننده آن را اجرا می‌کند. تمام ورودی/خروجی با `asyncio` و کاملاً ناهمزمان است: یک `DatagramProtocol` برای داده UDP و یک سرور TCP کوچک فقط برای فرمان‌های شبیه‌ساز (تزریق بسته و خاموش‌سازی). حالت نگهداری شده در هر گره:

- **NodeID**: یک UUID یکتا، کاملاً مستقل از آدرس شبکه (تولید قطعی از روی بذر).
- **SelfAddr**: زوج `ip:port` روی `127.0.0.1`؛ هر گره پورت UDP اختصاصی خود را دارد.
- **PeerList**: همسایه‌های مستقیم به همراه آدرس، تأخیر پایه لینک، آخرین زمان مشاهده (`last_seen`)، آخرین بسته دریافتی از آن همسایه و شمارنده ارسال/دریافت. درجه هر گره در بازه ۲ تا ۴ تضمین شده است.
- **SeenSet**: مجموعه شناسه بسته‌های پردازش‌شده برای حذف تکراری‌ها؛ مجموعه جداگانه `stem_seen` برای تشخیص بازگشت گشت تصادفی ساقه.

۱.۲ قالب بسته روی سیم

تنها بایتهایی که روی UDP منتقل می‌شوند عبارت‌اند از:

`<packet-id>|<S|F>` example: `p0137|S`

یعنی فقط شناسه یکتای بسته و وضعیت آن (Stem/Fluff). شناسه گره مبدأ و زمان تولید بسته هرگز منتقل نمی‌شوند و تنها در لاگ مرجع شبیه‌ساز ثبت می‌گردند. گره فرستنده قبلی صرفاً از روی آدرس مبدأ دیتاگرام UDP شناخته می‌شود — همان اطلاعاتی که یک گره در شبکه واقعی هم در اختیار دارد. ساعت همه گره‌ها همگام فرض شده و تأخیر پردازش داخلی صفر است؛ تنها در فاز ۵ گره‌های جاسوس تأخیر عمدی اعمال می‌کنند.

۱.۳ تولید توپولوژی

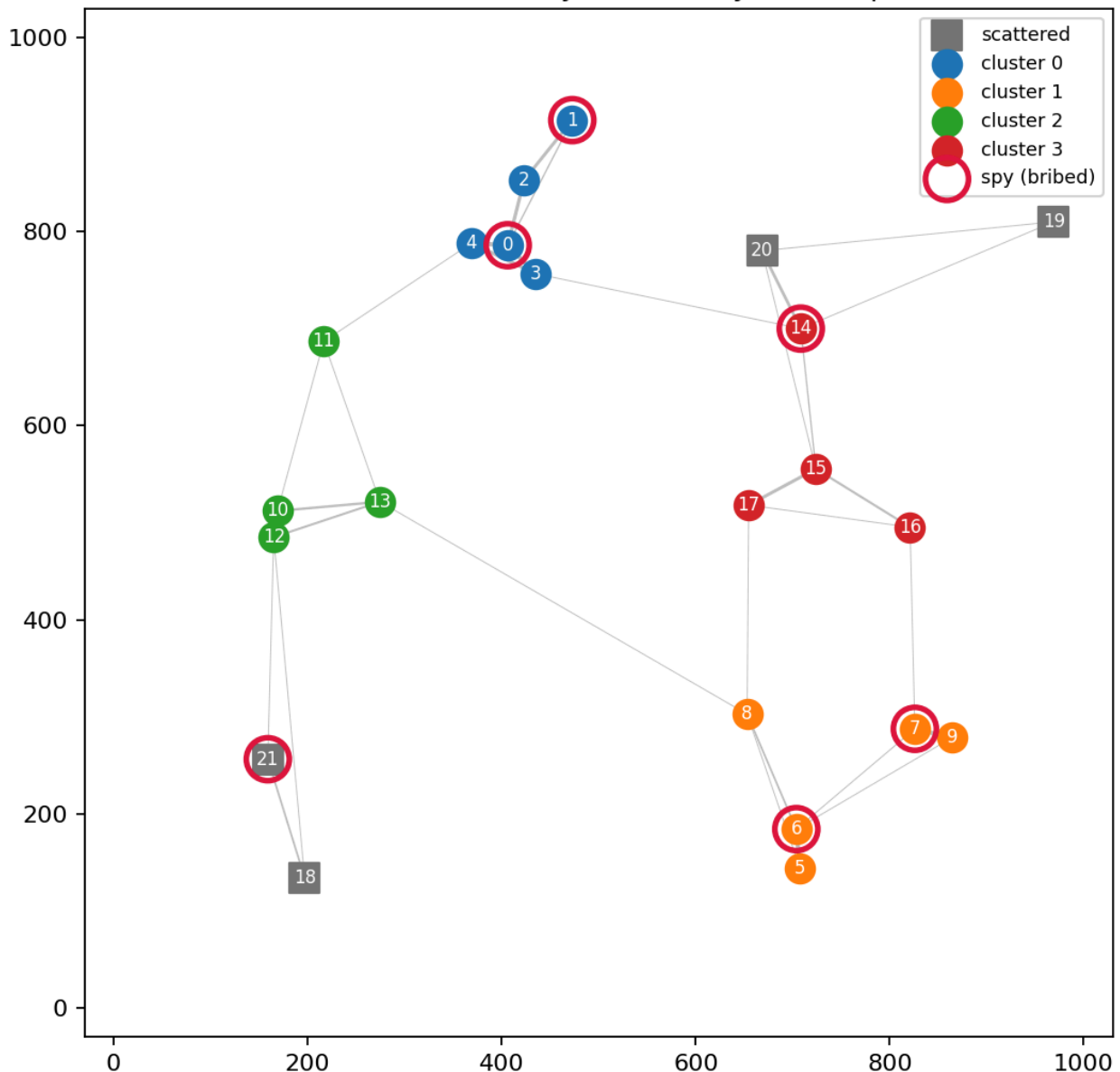
گره‌ها روی صفحه 1000×1000 جانمایی می‌شوند. الگوریتم به‌ترتیب زیر است:

1. **مراکز کلاستر:** 4 مرکز با نمونه‌گیری با پس‌زدن (rejection sampling) در ناحیه $[150, 850]^2$ انتخاب می‌شوند، با شرط حداقل فاصله 260 واحد میان هر دو مرکز (پخش تقریباً یکنواخت مراکز و جلوگیری از ادغام کلاسترها).
2. **اعضای کلاستر:** ۸۰٪ گره‌ها با توزیع گاوسی همسان‌گرد حول مرکز خود با $\sigma = 55$ نمونه‌گیری می‌شوند؛ نمونه‌ای که به مرکز دیگری نزدیکتر باشد یا بیرون صفحه بیفتد دور ریخته و دوباره نمونه‌گیری می‌شود. این کار تراکم موضعی زیاد و مرز روشن میان کلاسترها ایجاد می‌کند.
3. **گره‌های پراکنده:** ۲۰٪ باقی‌مانده به‌صورت یکنواخت روی صفحه و با حفظ حداقل فاصله 90 واحد از مراکز کلاستر قرار می‌گیرند.
4. **یال‌های درون‌کلاستری:** درخت پوشای کمینه اقلیدسی هر کلاستر ساخته می‌شود و سپس هر عضو با درجه کمتر از ۲ به نزدیکترین هم‌کلاستری خود وصل می‌شود.
5. **گره‌های پراکنده** به دو همسایه نزدیک خود (در کل شبکه) متصل می‌شوند.
6. **اتصال چندگانه کلاسترها:** مراکز کلاسترها بر حسب زاویه قطبی حول مرکز ثقل کل مرتب می‌شوند و یک حلقه میان کلاسترهای متوالی از طریق نزدیکترین زوج گره ساخته می‌شود؛ دو لینک هر کلاستر عمداً از دو گره متفاوت عبور می‌کنند تا مستقل باشند. سپس چند لینک عرضی کوتاه دیگر نیز افزوده می‌شود.
7. **ترمیم:** حلقه‌ای تکراری درجه همه گره‌ها را به بازه $[2, 4]$ می‌آورد، مؤلفه‌های جدا را وصل می‌کند و هر پل (یال برشی) را با افزودن کوتاه‌ترین وتر ممکن حذف می‌کند. اگر توپولوژی پس از ترمیم معتبر نباشد، با بذر مشتق‌شده دوباره تولید می‌شود.

چرا «بی‌پل بودن» را بررسی می‌کنیم؟ شرط صورت پروژه این است که هر کلاستر دستکم از دو لینک ارتباطی مستقل به بقیه شبکه وصل باشد تا مسیر جایگزین وجود داشته باشد. شرط قوی‌تر «گراف هیچ پلی ندارد» (۲-یال-همبندی) دقیقاً معادل این است که میان هر دو گره حداقل دو مسیر یال-مجزا وجود دارد؛ بنابراین برقراری آن، شرط صورت پروژه را برای همه کلاسترها و حتی همه گره‌ها تضمین می‌کند. اعتبارسنجی خودکار (تابع `topology.validate`) هر دو شرط را جداگانه گزارش می‌کند.

ویژگی	مقدار	ویژگی	مقدار
تعداد گره	22	تعداد یال	33
تعداد کلاستر	4	اندازه کلاسترها	5, 5, 4, 4
همبند است؟	بله	تعداد پل	0
درجه کمینه/بیشینه	2 / 4	درجه میانگین	3
لینک عرضی هر کلاستر	2, 3, 4, 6	شرط دو لینک مستقل	برای همه کلاسترها برقرار

Network topology (22 nodes, 4 clusters, 33 links)
link width ~ 1/base-delay, base delay = 1 ms per unit



شکل ۱ — توپولوژی ثابت همه آزمایش‌ها. رنگ‌ها کلاسترها را نشان می‌دهند، مربع‌های خاکستری گره‌های پراکنده و دایره‌های قرمز گره‌های جاسوس‌اند. ضخامت یال‌ها معکوس تأخیر پایه است.

۱.۴ مدل تأخیر لینک

تأخیر پایه هر لینک برابر فاصله اقلیدسی دو سر آن است (۱ میلی‌ثانیه به ازای هر واحد فاصله) و به ازای هر ارسال جیت‌ر یکنواخت ۲۰٪ اعمال می‌شود:

$$\text{Delay}_{\text{total}} = \text{Delay}_{\text{base}} + U(-0.2 \times \text{Delay}_{\text{base}}, +0.2 \times \text{Delay}_{\text{base}})$$

پیدامساز: فرستنده تأخیر را محاسبه و ارسال واقعی دیتاگرام را با `loop.call_later` زمان‌بندی می‌کند؛ بنابراین تأخیر لینک بدون مسدود کردن حلقه رویداد مدل می‌شود. اسکریپت `src/verify.py` روی همه ۴۰ اجرا بررسی می‌کند که نسبت تأخیر زمان‌بندی‌شده به تأخیر پایه دقیقاً در بازه $[0.8, 1.2]$ بماند (و برای جاسوس‌های فاز ۵ در $[1.8, 2.2]$).

۱.۵ مدل رفتاری مهاجم

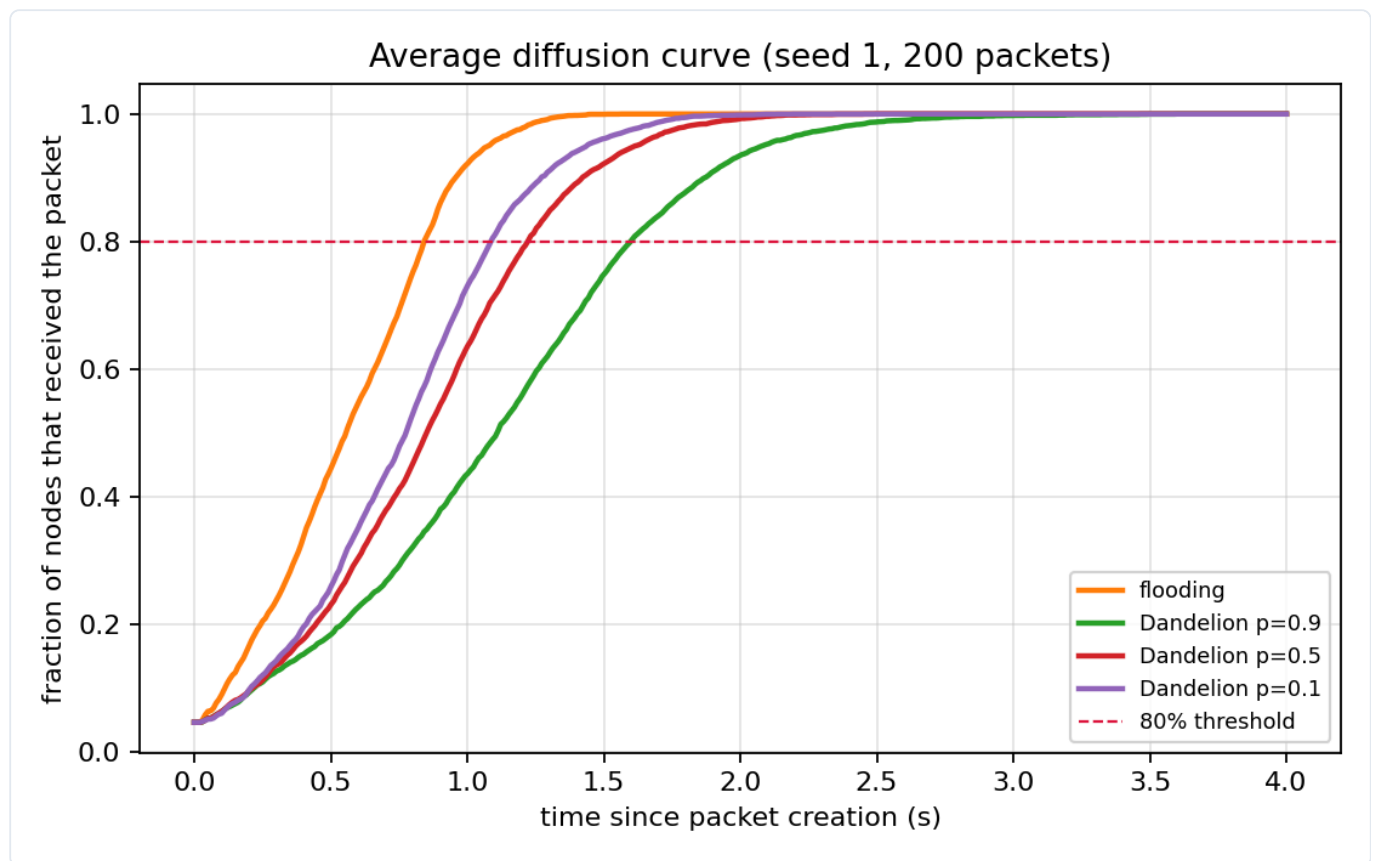
- مهاجم گره جدید اضافه نمی‌کند؛ تنها با رشوه، گره‌های موجود را جاسوس می‌کند و هزینه رشوه برای همه گره‌ها یکسان است — به همین دلیل امتیاز او بر تعداد جاسوس‌ها تقسیم می‌شود.
- مهاجم تأخیر پایه همه لینک‌ها و توپولوژی را می‌داند، اما دید سراسری ندارد؛ فقط بسته‌هایی را می‌بیند که به جاسوس‌های او می‌رسند (شناسه بسته، وضعیت، همسایه تحویل‌دهنده و زمان ورود محلی).
- جاسوس‌ها حق حذف یا مسدود کردن بسته را ندارند و دقیقاً طبق پروتکل بازپخش می‌کنند؛ تنها در فاز ۵ می‌توانند پیش از بازپخش تأخیر عمدی (حداکثر یک تأخیر پایه) اعمال کنند.
- گم‌شدن بسته در لایه UDP مدل نمی‌شود؛ همه ارسال‌های محلی موفق فرض می‌شوند و هر رویداد غیرمنتظره در لاگ ثبت و توسط verify بررسی می‌شود (در هیچ اجرایی رخ نداد).

۲. سناریوی آزمایش

در هر اجرا 200 بسته تولید می‌شود. مبدأ هر بسته به‌صورت کاملاً تصادفی از میان گره‌های درستکار انتخاب می‌شود و زمان تولید بسته‌ها از توزیع یکنواخت روی یک پنجره زمانی (بدون هیچ الگوی زمانی از پیش تعیین‌شده) کشیده می‌شود. زمان‌بندی و مبدأها تنها از روی بذر سناریو ساخته می‌شوند؛ بنابراین برای یک بذر مشخص، دقیقاً همان ۲۰۰ زوج (مبدأ، زمان) در همه فازها و همه مقادیر p تکرار می‌شود و مقایسه منصفانه است. توپولوژی و مجموعه جاسوس‌ها نیز در کل کارزار ثابت‌اند. پنج بذر مستقل [1, 2, 3, 4, 5] استفاده شده و میانگین، میانه و انحراف معیار گزارش می‌شود.

۳. فاز ۱ — پخش عمومی پایه

هر گره پس از نخستین دریافت یک بسته، بلافاصله آن را برای همه همسایه‌های خود ارسال می‌کند و نسخه‌های تکراری با SeenSet دور ریخته می‌شوند. نتیجه: پوشش کامل (100.0% گره‌ها در همه اجراها)، 66.0 دیتاگرام به ازای هر بسته (دقیقاً دو برابر تعداد یال‌ها، همان‌طور که برای پخش سیل‌آسا انتظار می‌رود) و زمان پوشش ۸۰٪ برابر 0.835 ± 0.006 ثانیه.



شکل ۲ — منحنی میانگین انتشار: نسبت گره‌هایی که بسته را دریافت کرده‌اند بر حسب زمان از لحظه تولید بسته.

۴. فاز ۲ — حمله به پخش عمومی

۴.۱ خط پایه: «اولین جاسوس مشاهده‌کننده»

در این روش، مهاجم نخستین مشاهده ثبت‌شده در میان جاسوس‌ها را برمی‌دارد و همسایه‌ای که بسته را به آن جاسوس تحویل داده است را مبدأ حدس می‌زند (حدس زدن خود جاسوس بی‌معناست، چون مبدأها همیشه از گره‌های درستکار انتخاب می‌شوند). اگر آن همسایه خود جاسوس باشد، نخستین مشاهده‌ای که فرستنده‌اش درستکار است استفاده می‌شود.

۴.۲ روش پیشنهادی: تخمین بیشینه‌درست‌نمایی زمانی

مهاجم تأخیر پایه همه لینک‌ها را می‌داند، پس می‌تواند کوتاه‌ترین تأخیر مسیر $d(v, s)$ را میان هر گره نامزد v و هر جاسوس s با دایکسترا محاسبه کند. اگر بسته در لحظه نامعلوم t_0 در v ساخته شده باشد، زمان نخستین ورود آن به جاسوس s برابر است با

$$t_s = t_0 + d(v, s) + \varepsilon_s, \text{Var}[\varepsilon_s] = (0.2^2/3) \cdot \sum_{i \in \text{path}} d_i^2$$

چون جیتز هر لینک یکنواخت و مستقل است، واریانس خطا مجموع واریانس جیتز یال‌های مسیر است. با حذف پارامتر مزاحم t_0 (کمینه‌سازی بسته مربعات وزندار)، امتیاز هر نامزد به‌دست می‌آید:

$$\chi^2(v) = \sum_s w_s (t_s - d(v, s) - \hat{t}_0(v))^2, w_s = 1 / \text{Var}[\varepsilon_s], \hat{t}_0(v) = \sum w_s (t_s - d(v, s)) / \sum w_s$$

فیلتر سازگاری هاپ آخر. علاوه بر زمان، مهاجم می‌داند بسته از کدام همسایه به هر جاسوس رسیده است. در پخش عمومی، نخستین نسخه رسیده تقریباً همیشه از کوتاه‌ترین مسیر می‌آید؛ بنابراین برای مبدأ واقعی باید $d(v, u) + \text{delay}(u, s) \approx d(v, s)$ برقرار باشد. هر نامزدی که این شرط را (با حاشیه ۲۵٪) نقض کند جریمه بزرگی می‌گیرد. این فیلتر عملاً هذلولی‌های هم‌ارز در تخمین زمانی را می‌شکند و بیشترین سهم را در بهبود دقت دارد. اگر تنها یک جاسوس بسته را ببیند، تفاضل زمانی معنا ندارد و روش به همان حدس همسایه تحویل‌دهنده بازمی‌گردد.

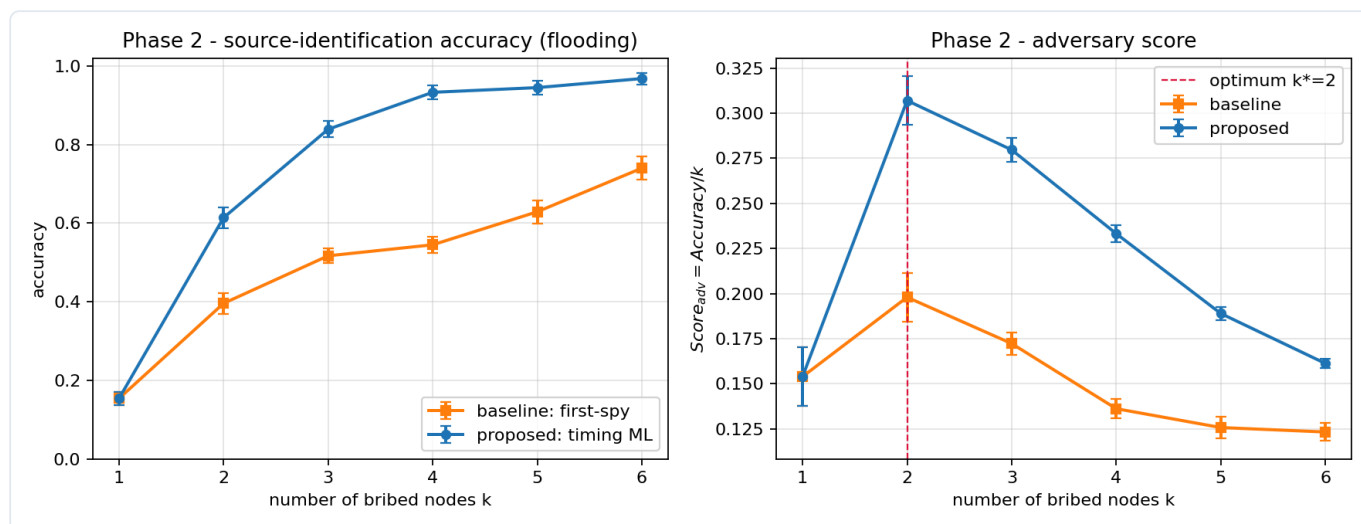
۴.۳ مکان‌یابی و تعداد بهینه جاسوس‌ها

چون $\text{Score}_{\text{adv}} = \text{Accuracy} / k$ است، افزودن جاسوس تنها وقتی به‌صرفه است که دقت را بیش از نسبت $k/(k-1)$ بالا ببرد. برای انتخاب مکان‌ها، پنج راهبرد ابتکاری (k -center، بیشترین درجه، بیشترین بینابینی، ترکیبی و تصادفی) و یک جستجوی حریصانه رو به جلو روی دو اجرای کالیبراسیون مستقل (بذره‌های [101, 102]، جدا از پنج بذر ارزیابی) مقایسه شدند. جستجوی حریصانه در هر گام گره‌ای را می‌افزاید که بیشترین افزایش دقت را بدهد؛ نتیجه آن تودرتو است، یعنی مجموعه بهینه k جاسوس زیرمجموعه مجموعه $k+1$ جاسوس است:

ترتیب انتخاب جاسوس‌ها: $0 \rightarrow 7 \rightarrow 1 \rightarrow 21 \rightarrow 14 \rightarrow 6$

Score _{adv}	دقت روش پیشنهادی	دقت خط پایه	جاسوس‌ها	k
0.1540	0.154 ± 0.016	0.154 ± 0.016	6	1
0.3070	0.614 ± 0.027	0.396 ± 0.027	6, 14	2 ★
0.2797	0.839 ± 0.020	0.517 ± 0.018	6, 14, 21	3
0.2333	0.933 ± 0.018	0.545 ± 0.021	1, 6, 14, 21	4
0.1890	0.945 ± 0.018	0.629 ± 0.029	1, 6, 7, 14, 21	5
0.1613	0.968 ± 0.014	0.740 ± 0.029	0, 1, 6, 7, 14, 21	6

بیشینه Score_{adv} در $k^* = 2$ رخ می‌دهد (مقدار 0.3070). با این حال برای اینکه مقایسه فازها در همه نقاط کاری قابل قضاوت باشد، همه جدول‌ها برای $k = 1$ تا $k = 6$ (سقف ۳۰٪ رشوه) گزارش شده‌اند و شبیه‌سازی‌ها با مجموعه کامل ۶ جاسوس اجرا شده‌اند. نکته مهم این است که در فازهای ۱ تا ۴ رفتار جاسوس دقیقاً مانند گره درستکار است، پس مجموعه جاسوس‌ها هیچ اثری بر دینامیک شبکه ندارد و ارزیابی هر زیرمجموعه از روی لاگ مرجع دقیق است، نه تقریبی.



شکل ۳ — چپ: دقت تشخیص مبدأ بر حسب تعداد جاسوس. راست: امتیاز مهاجم Accuracy/k و نقطه بهینه.

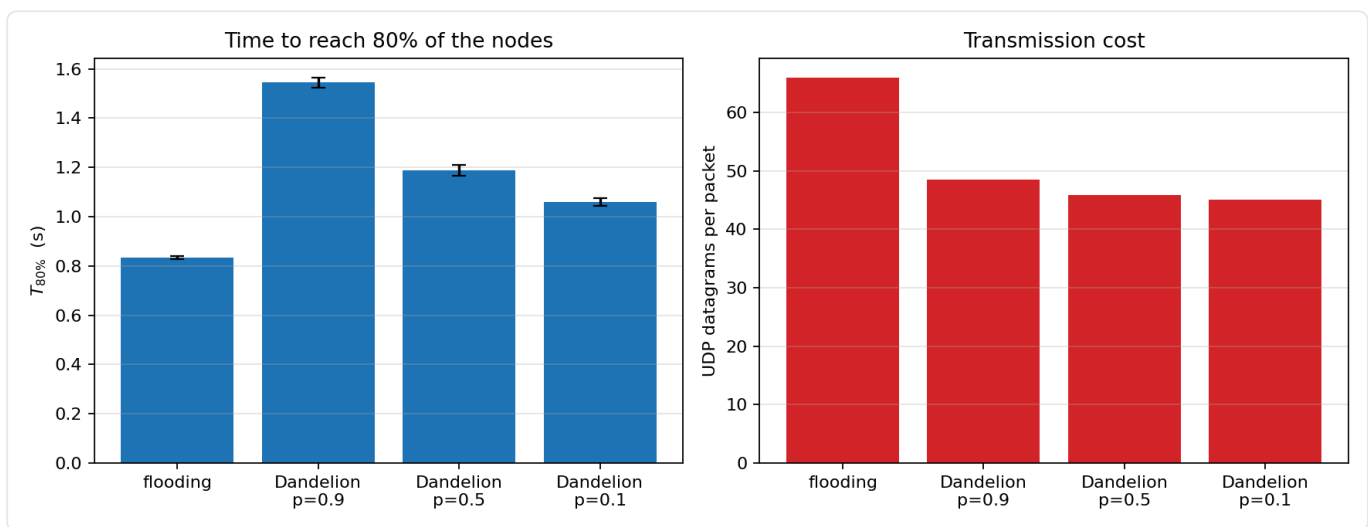
۵. فاز ۳ — پیاده‌سازی Dandelion و جاروب پارامتر p

ماشین حالت پیاده‌سازی شده دقیقاً مطابق صورت پروژه است:

- گره مبدأ همیشه بسته را در وضعیت ساقه می‌سازد و فقط برای یک همسایه تصادفی می‌فرستد.
- هر گره دریافت‌کننده مستقلاً با احتمال p بسته را در وضعیت ساقه نگه می‌دارد و آن را برای یکی از همسایه‌های خود به‌جز فرستنده قبلی می‌فرستد؛ اگر همسایه واجد شرایطی نباشد، بسته وارد وضعیت پرز می‌شود.
- با احتمال $p - 1$ بسته وارد وضعیت پرز می‌شود و از آن پس تا پایان انتشار در همین وضعیت می‌ماند: هر گره پس از نخستین دریافت، بسته را برای همه همسایه‌ها به‌جز فرستنده قبلی ارسال می‌کند و تکراری‌ها با SeenSet نادیده گرفته می‌شوند.

یک تصمیم طراحی که در صورت پروژه مسکوت است. گشت تصادفی ساقه روی گراف متناهی ممکن است به گره‌ای برگردد که همان بسته را قبلاً در وضعیت ساقه دیده است. اگر چنین بسته‌ای مثل نسخه تکراری دور ریخته شود، بسته برای همیشه گم می‌شود. بنابراین قاعده زیر را گذاشتیم: بازگشت گشت ساقه به گره تکراری، بسته را بلافاصله وارد وضعیت پرز می‌کند. این کار خاتمه قطعی و پوشش کامل را تضمین می‌کند (در همه اجراها 100.0% پوشش) و در لاگ با رویداد stem_loop ثبت می‌شود.

پروتکل	میانگین $T_{80\%}$	میان	انحراف معیار	دیتاگرام بر بسته	طول متوسط ساقه
پخش عمومی	0.835 s	0.834	0.006	66.0	—
Dandelion $p=0.9$	1.545 s	1.506	0.021	48.5	4.46
Dandelion $p=0.5$	1.189 s	1.119	0.022	45.9	1.89
Dandelion $p=0.1$	1.061 s	1.005	0.015	45.1	1.11



شکل ۴ — زمان پوشش ۸۰٪ و هزینه ارسال برای پخش عمومی و سه مقدار p .

۶. فاز ۴ — حمله پیشرفته به Dandelion

حمله فاز ۲ در برابر Dandelion کارایی کافی ندارد، زیرا موج پرز از گره پایان ساقه شروع می‌شود، نه از مبدأ؛ پس تخمین زمانی، محل «شروع پرز» را پیدا می‌کند نه مبدأ واقعی را. حمله پیشنهادی از دو منبع شواهد استفاده می‌کند:

۶.۱ هسته گشت تصادفی معکوس (شواهد ساقه)

اگر یکی از جاسوس‌ها بسته را در وضعیت ساقه ببیند، می‌داند گشت ساقه از یال $(u \rightarrow \text{spy})$ گذشته است. گشت ساقه یک زنجیره مارکوف روی یال‌های جهت‌دار است (چون هرگز به فرستنده قبلی برنمی‌گردد):

$$T[(a \rightarrow b), (b \rightarrow c)] = p / (\deg(b) - 1), c \neq a, M = \sum_{k \geq 0} T^k = (I - T)^{-1}$$

درست‌نمایی اینکه مبدأ گره v باشد برابر است با احتمال اینکه گشتی که از v شروع شده (مبدأ همسایه اول را یکنواخت از میان $\deg(v)$ همسایه برمی‌دارد) به یال مشاهده‌شده برسد:

$$L(v) = (1 / \deg(v)) \cdot \sum_{w \in N(v)} M[(v \rightarrow w), (u \rightarrow \text{spy})]$$

چون $\rho(T) \leq p < 1$ است، سری همگراست و ماتریس M فقط یک بار به ازای هر p محاسبه می‌شود (اندازه آن برابر تعداد یال‌های جهت‌دار است). این هسته به‌طور خودکار سه اثر را با هم در نظر می‌گیرد: نمایی افت‌کردن احتمال با طول ساقه، اثر درجه گره‌ها بر انتخاب مسیر، و تعداد مسیرهای موجود میان دو گره.

۶.۲ هسته پایان ساقه (شواهد پرز)

اگر هیچ جاسوسی ساقه را نبیند، مهاجم ابتدا با همان تخمین‌گر زمانی فاز ۲ توزیع احتمال روی گره شروع پرز را می‌سازد ($P_{\text{time}}(f) \propto \exp(-\chi^2(f)/2)$ به‌همراه جریمه ناسازگاری هاپ آخر) و سپس آن را از هسته پایان ساقه عبور می‌دهد:

$$K[v, f] = P(v) = \sum_f P_{\text{time}}(f) \cdot K[v, f]$$

هنگامی که شواهد ساقه موجود باشد، شواهد پرز اطلاعات مستقلی درباره مبدأ اضافه نمی‌کند: هرچه پس از جاسوس رخ می‌دهد برای مهاجم کاملاً معلوم است، پس ضرب این دو درست‌نمایی، شواهد هم‌بسته را دوباره می‌شمارد. آزمایش هم همین را نشان داد (ترکیب کورکورانه دقت را در $p = 0.5$ حدود ۴ واحد درصد پایین آورد): بنابراین تخمین‌گر نهایی هرگاه ساقه دیده شود فقط از هسته معکوس استفاده می‌کند و در غیر این صورت به شواهد پرز تکیه می‌کند.

۶.۳ نتایج

$$p = 0.9$$

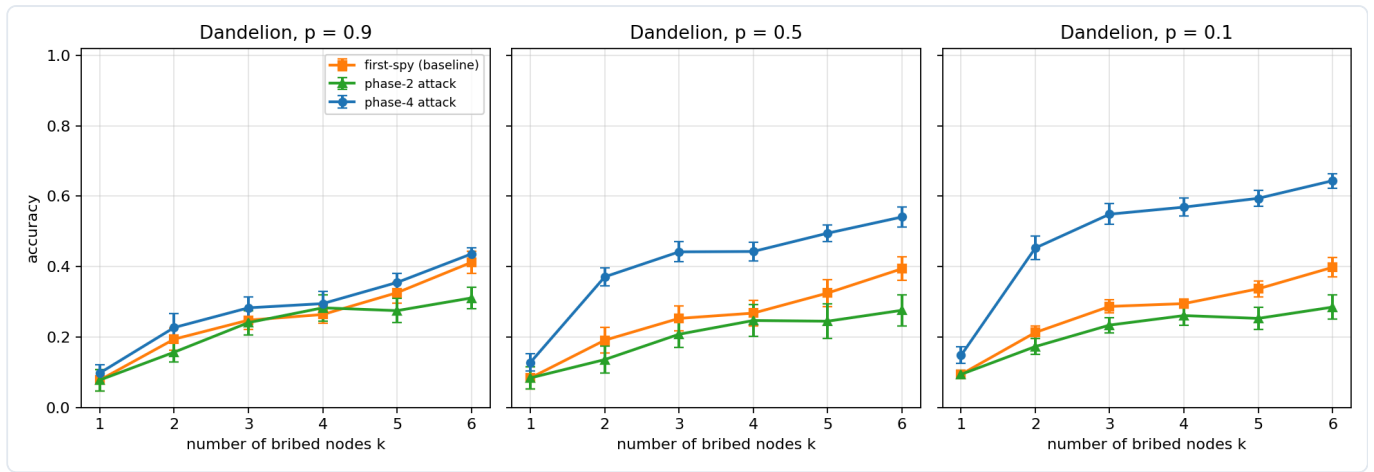
خط پایه (اولین جاسوس)	حمله فاز ۲	حمله فاز ۴	Score _{adv} (فاز ۴)	k
0.078 ± 0.031	0.078 ± 0.031	0.098 ± 0.023	0.0980	1
0.194 ± 0.032	0.157 ± 0.027	0.227 ± 0.039	0.1135	2
0.248 ± 0.027	0.241 ± 0.035	0.283 ± 0.031	0.0943	3
0.264 ± 0.024	0.283 ± 0.037	0.295 ± 0.035	0.0738	4
0.326 ± 0.029	0.275 ± 0.035	0.355 ± 0.025	0.0710	5
0.412 ± 0.032	0.311 ± 0.030	0.436 ± 0.018	0.0727	6

$p = 0.5$

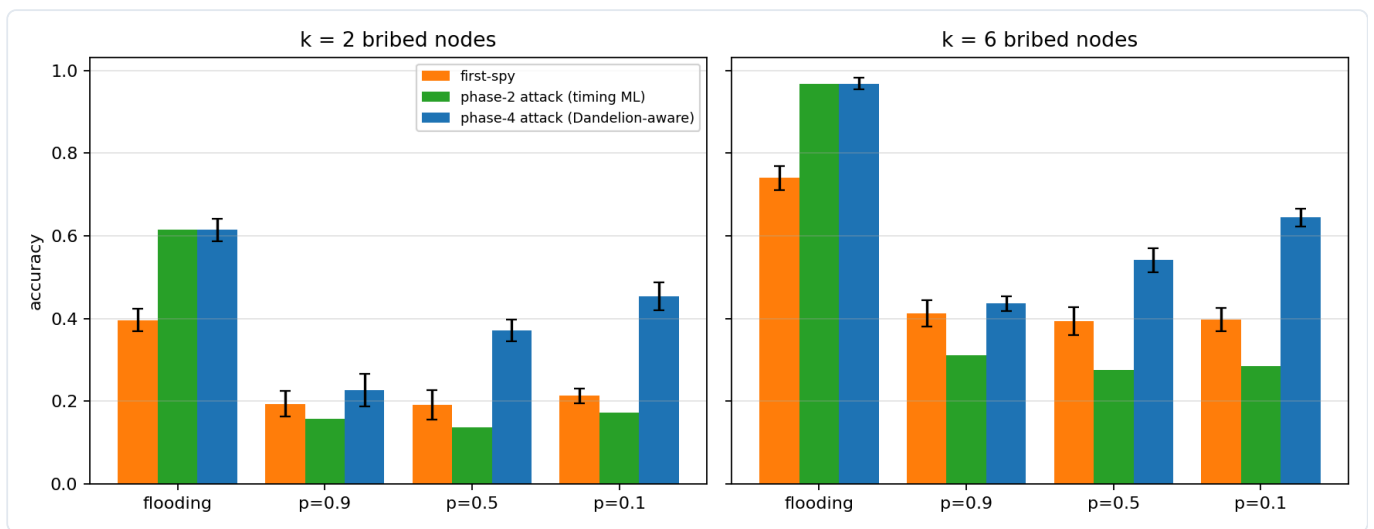
خط پایه (اولین جاسوس)	حمله فاز ۲	حمله فاز ۴	Score _{adv} (فاز ۴)	k
0.084 ± 0.032	0.084 ± 0.032	0.128 ± 0.024	0.1280	1
0.191 ± 0.036	0.136 ± 0.039	0.371 ± 0.026	0.1855	2
0.253 ± 0.035	0.208 ± 0.037	0.442 ± 0.028	0.1473	3
0.268 ± 0.036	0.247 ± 0.045	0.443 ± 0.026	0.1107	4
0.325 ± 0.039	0.245 ± 0.050	0.495 ± 0.023	0.0990	5
0.394 ± 0.034	0.276 ± 0.045	0.541 ± 0.029	0.0902	6

$p = 0.1$

خط پایه (اولین جاسوس)	حمله فاز ۲	حمله فاز ۴	Score _{adv} (فاز ۴)	k
0.094 ± 0.011	0.094 ± 0.011	0.148 ± 0.024	0.1480	1
0.213 ± 0.018	0.173 ± 0.023	0.453 ± 0.034	0.2265	2
0.287 ± 0.019	0.234 ± 0.022	0.549 ± 0.029	0.1830	3
0.295 ± 0.012	0.261 ± 0.028	0.569 ± 0.026	0.1423	4
0.337 ± 0.023	0.253 ± 0.031	0.594 ± 0.022	0.1188	5
0.398 ± 0.028	0.285 ± 0.034	0.644 ± 0.021	0.1073	6



شکل ۵ — دقت سه تخمین‌گر در برابر Dandelion برای سه مقدار p .



شکل ۶ — مقایسه مستقیم حمله فاز ۲ و فاز ۴ روی پخش عمومی و سه مقدار p .

۶.۴ آیا مکان بهینه جاسوس‌ها به پروتکل وابسته است؟

جستجوی حریصانه را این بار مستقیماً روی اجراهای $p = 0.5$ Dandelion، بذره‌های [1, 2] تکرار کردیم و هر دو مجموعه را روی بذره‌های کنارگذاشته‌شده [3, 4, 5] ارزیابی کردیم تا نتیجه از بیش‌برازش پاک باشد:

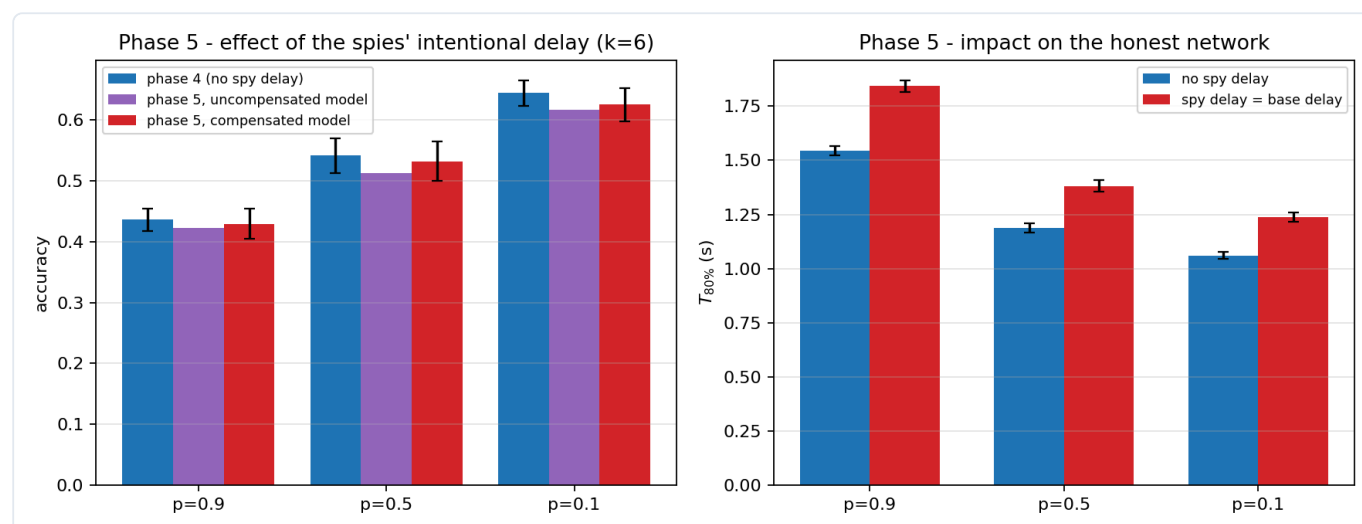
k	مجموعه بهینه‌شده برای پخش عمومی	دقت (بذره‌های کنارگذاشته)	مجموعه بهینه‌شده برای Dandelion	دقت (بذره‌های کنارگذاشته)
1	6	0.120 ± 0.030	8	0.209 ± 0.038
2	6, 14	0.375 ± 0.026	8, 14	0.366 ± 0.007
3	6, 14, 21	0.440 ± 0.035	8, 12, 14	0.431 ± 0.016
4	1, 6, 14, 21	0.440 ± 0.036	8, 12, 14, 15	0.542 ± 0.047
5	1, 6, 7, 14, 21	0.493 ± 0.030	8, 11, 12, 14, 15	0.653 ± 0.045
6	0, 1, 6, 7, 14, 21	0.530 ± 0.035	0, 8, 11, 12, 14, 15	0.702 ± 0.041

پاسخ روشن است: **بله**. تا $k = 3$ دو مجموعه عملاً هم‌ارزند، اما از آن به بعد مکان‌یابی مخصوص Dandelion جلو می‌افتد و در $k = 6$ حدود 17 واحد درصد دقت بیشتری می‌دهد (70.2% در برابر 53.0%). دلیل شهودی: تخمین‌گر زمانی به **هندسه خوب مثلث‌بندی** نیاز دارد و جاسوس‌های دور از هم را می‌پسندد، در حالی که حمله Dandelion باید **کشت ساقه را قطع کند** و بنابراین گره‌هایی با درجه و بینابینی بالا را ترجیح می‌دهد. برای اینکه مقایسه فازها منصفانه بماند، همه اعداد بخش‌های قبل با یک مجموعه جاسوس ثابت (بهینه‌شده در فاز ۲) گزارش شده‌اند؛ بنابراین ارقام فاز ۴ یک **کران پایین** از توان واقعی مهاجم‌اند.

۷. فاز ۵ — تأخیر عمدی جاسوس‌ها

در این فاز هر جاسوس پیش از بازپخش، تأخیر عمدی به اندازه یک تأخیر پایه همان لینک (بیشینه مجاز) اضافه می‌کند؛ حذف بسته یا نگهداری نامحدود مجاز نیست. دو مدل مهاجم را مقایسه می‌کنیم: مدل جبران‌نشده که همچنان تأخیر پایه را برای همه لینک‌ها فرض می‌کند، و مدل جبران‌شده که می‌داند هزینه هر لینک خروجی از یک جاسوس دو برابر شده است و کوتاه‌ترین مسیرها را روی گراف جهت‌دار اصلاح‌شده حساب می‌کند.

سناریو	دقت بدون تأخیر (فاز ۴)	دقت با تأخیر — مدل جبران‌نشده	دقت با تأخیر — مدل جبران‌شده	بدون $T_{80\%}$ تأخیر	با $T_{80\%}$ تأخیر
Dandelion $p=0.9$	0.436 ± 0.018	0.422 ± 0.017	0.429 ± 0.025	1.545 s	1.842 s (+19%)
Dandelion $p=0.5$	0.541 ± 0.029	0.512 ± 0.028	0.532 ± 0.032	1.189 s	1.382 s (+16%)
Dandelion $p=0.1$	0.644 ± 0.021	0.617 ± 0.024	0.625 ± 0.027	1.061 s	1.237 s (+17%)
پخش عمومی	0.968 ± 0.014	0.861 ± 0.032	0.968 ± 0.007	0.835 s	0.968 s (+16%)



شکل ۷ — اثر تأخیر عمدی جاسوس‌ها بر دقت مهاجم (چپ) و بر زمان پوشش شبکه (راست).

تحلیل — آیا تأخیر به نفع مهاجم است؟

(۱) اثر بر دقت: عملاً هیچ. دقت مشترک مهاجم به‌طور متوسط ۱.۲- واحد درصد کاهش می‌یابد، در حالی که انحراف معیار میان بذرها حدود ۲.۸ واحد درصد است؛ یعنی تغییر در حد نوسان آماری است و تأخیر عمدی، مهاجم را دقیق‌تر نمی‌کند.

(۲) اما مدل‌سازی درست حیاتی است. اگر مهاجم تأخیر خودش را در کوتاه‌ترین مسیرها لحاظ نکند، بخشی از دقتش را از دست می‌دهد؛ این اثر در پخش عمومی چشمگیر است: ۸۶.۱٪ با مدل جبران‌نشده در برابر ۹۶.۸٪ با مدل جبران‌شده — یعنی مهاجم

آگاه، تمام زیان ناشی از تأخیر خودش را بازمی‌گرداند. علت روشن است: تأخیر عمدی برای مهاجم اطلاعات پنهان نیست؛ او دقیقاً می‌داند روی کدام لینک چقدر صبر کرده و کافی است هزینه یال‌های خروجی از جاسوس‌ها را دو برابر کند و مسیرها را روی گراف جهت‌دار اصلاح‌شده حساب کند.

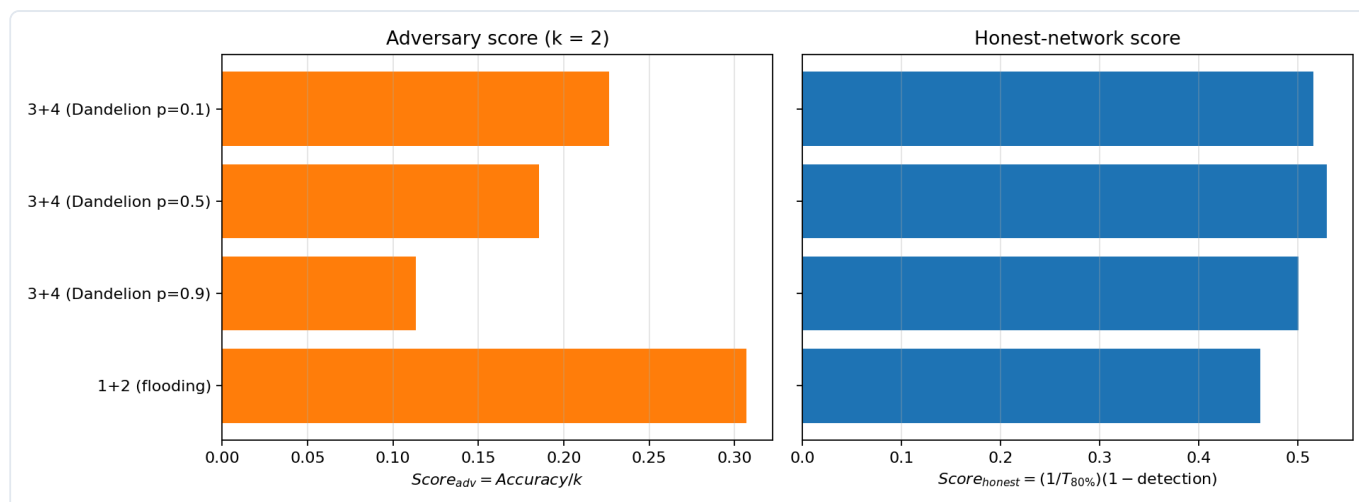
(۳) **اثر واقعی روی شبکه درستکار است.** زمان رسیدن بسته به ۸۰٪ گره‌ها به‌طور متوسط ۱۷٪+ بزرگتر می‌شود و در نتیجه $Score_{honest}$ حدود ۱۲٪- تغییر می‌کند. بنابراین تأخیر عمدی نه یک ابزار «افزایش دقت»، بلکه یک **حمله کیفیت سرویس** است: مهاجم بدون پرداخت هیچ هزینه‌ای در دقت خود، امتیاز شبکه درستکار را پایین می‌آورد. اگر معیار دآوری تنها $Score_{adv}$ باشد، این کار برای مهاجم بی‌تفاوت است؛ اما اگر بازی مجموع‌صفر میان دو امتیاز در نظر گرفته شود، راهبرد غالب مهاجم است.

(۴) **چرا ترتیب مشاهددها به هم نمی‌ریزد؟** بیشینه تأخیر مجاز برابر یک تأخیر پایه همان لینک است. چون بسته‌ها معمولاً از چند مسیر موازی به جاسوس‌ها می‌رسند و بخش عمده مسیرها از گره‌های درستکار عبور می‌کند، فقط بخشی از مشاهددها جابه‌جا می‌شوند و تخمین‌گر بیشینه‌درست‌نمایی — که همه جاسوس‌ها را هم‌زمان و به‌صورت وزندار در نظر می‌گیرد — نسبت به این جابه‌جایی مقاوم است. خط پایه «اولین جاسوس» که فقط به یک مشاهده تکیه دارد، آسیب‌پذیرتر است.

۸. معیارهای نهایی

$$Score_{adv} = Accuracy / (Number \text{ of } \text{Bribed Nodes}) \quad Score_{honest} = (1 / T_{80\%}) \times (1 - Adversary \text{ Detection Rate})$$

Score _{honest}	Score _{adv}	T _{80%}	دقت خط پایه	دقت (روش پیشنهادی)	سناریو	k
0.4621	0.3070	0.835 s	39.6%	61.4%	(flooding) 1+2	2
0.5004	0.1135	1.545 s	19.4%	22.7%	(Dandelion p=0.9) 3+4	2
0.5292	0.1855	1.189 s	19.1%	37.1%	(Dandelion p=0.5) 3+4	2
0.5157	0.2265	1.061 s	21.3%	45.3%	(Dandelion p=0.1) 3+4	2
0.0383	0.1613	0.835 s	74.0%	96.8%	(flooding) 1+2	6
0.3651	0.0727	1.545 s	41.2%	43.6%	(Dandelion p=0.9) 3+4	6
0.3862	0.0902	1.189 s	39.4%	54.1%	(Dandelion p=0.5) 3+4	6
0.3356	0.1073	1.061 s	39.8%	64.4%	(Dandelion p=0.1) 3+4	6
0.3099	0.0715	1.842 s	40.9%	42.9%	(Dandelion p=0.9 + spy delay) 5	6
0.3387	0.0887	1.382 s	40.6%	53.2%	(Dandelion p=0.5 + spy delay) 5	6
0.3031	0.1042	1.237 s	40.3%	62.5%	(Dandelion p=0.1 + spy delay) 5	6



شکل ۸ — امتیاز مهاجم و امتیاز شبکه درستکار.

۹. جمع‌بندی

1. در پخش عمومی، تخمین‌گر زمانی پیشنهادی دقت خط پایه را در $k = 6$ از 74.0% به 96.8% می‌رساند — یعنی صرفِ دانستن تأخیر پایه لینک‌ها، حتی با کمتر از یک‌سوم شبکه، حریم خصوصی مبدأ را عملاً از بین می‌برد.
2. Dandelion دفاع مؤثری است: در $p = 0.9$ دقت بهترین حمله ما با 6 جاسوس به 43.6% می‌افتد. هزینه آن افزایش 85% در زمان پوشش است.
3. پارامتر p دقیقاً همان مبادله «حریم خصوصی در برابر تأخیر» است: از $p = 0.1$ تا $p = 0.9$ دقت مهاجم از 64.4% به 43.6% کاهش و T_{80} از $s = 1.061$ به $s = 1.545$ افزایش می‌یابد.
4. حمله آگاه از ساختار (هسته گشت معکوس) در همه مقادیر p از خط پایه بهتر یا برابر است و بیشترین برتری را در p کوچک و متوسط نشان می‌دهد، جایی که ساقه کوتاه است و شواهد پرز قابل بهره‌برداری‌اند.
5. تأخیر عمدی جاسوس‌ها ابزار خوبی برای مهاجم است، اما نه به‌خاطر افزایش دقت؛ بلکه چون امتیاز شبکه درستکار را از راه بزرگ‌کردن T_{80} پایین می‌آورد.

۱۰. بازتولید نتایج

```
cd Code
python3 -m pip install -r requirements.txt
python3 -m src.experiments all          # توپولوژی + کالبراسیون + فاز ۱ تا ۵ + تحلیل
python3 -m src.plots                    # نمودارها و جدول‌ها
python3 -m src.verify                    # بررسی خودکار صحت روی همه اجراها
python3 -m src.report                    # ساخت همین گزارش از روی نتایج
python3 -m src.demo --mode dandelion --p 0.5 --packets 20 # اجرای نمایشی
```

هر اجرا پوشه خود را در `results/runs/` می‌سازد و در صورت وجود، دوباره اجرا نمی‌شود؛ بنابراین کارزار قابل توقف و ادامه است. بذر سناریو، توپولوژی، مجموعه جاسوس‌ها و مبدأ و زمان دقیق هر بسته در `meta.json` هر اجرا ذخیره می‌شود.

۱۱. محدودیت‌ها و تقریب‌ها

- هسته گشت معکوس، قاعده «شکستن حلقه» را مدل نمی‌کند (گشت‌هایی که به گره تکراری برمی‌گردند زودتر پرز می‌شوند). این تقریب تنها در p بزرگ محسوس است و تخمین‌گر را محافظه‌کارانه‌تر می‌کند.
- زمان‌بندی روی یک ماشین واقعی انجام می‌شود؛ سرِبار حلقه رویداد و زمان‌بند سیستم‌عامل به تأخیر مدل‌شده اضافه می‌شود. اندازه‌گیری‌های `verify.py` نشان می‌دهد میانه این سرِبار حدود ۱ میلی‌ثانیه و صدک ۹۹ آن حدود ۴ میلی‌ثانیه است، در برابر تأخیر پایه لینک‌ها که ده‌ها تا صدها میلی‌ثانیه است.
- تخمین‌گر زمانی فرض می‌کند نخستین نسخه رسیده از کوتاه‌ترین مسیر آمده است؛ با جیتر ۲۰٪ این فرض در اکثر موارد درست است و فیلتر سازگاری هاپ آخر موارد نقض را حذف می‌کند.